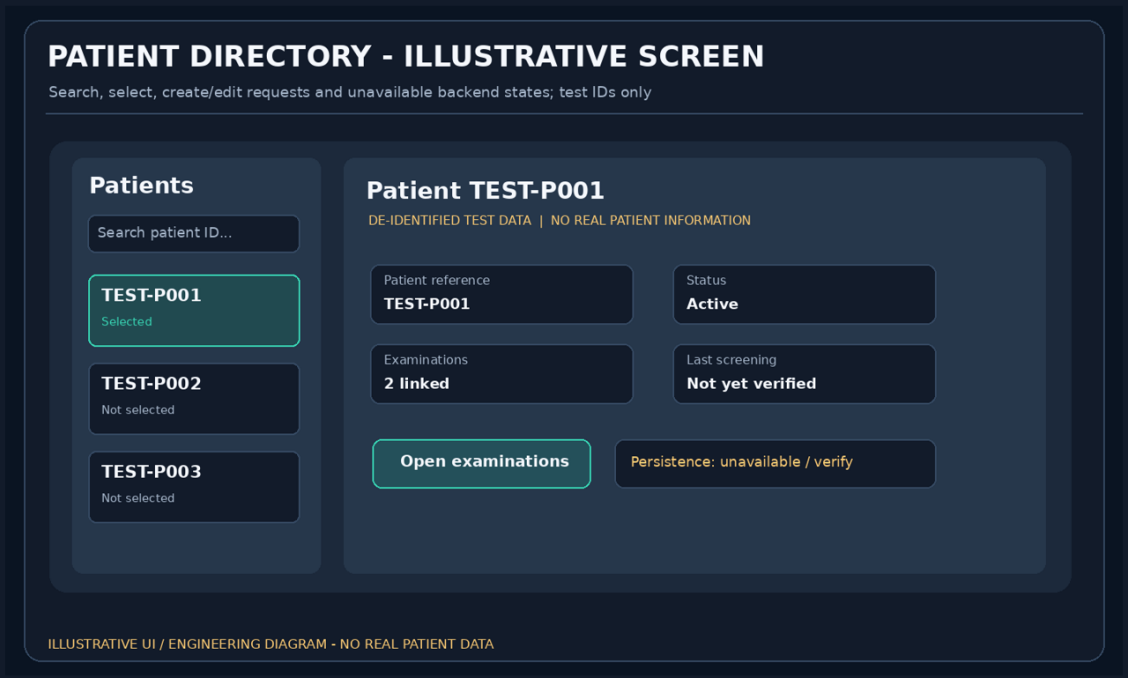
RETINACARE / GUI PHASE 1

# TASK 06

## Patient & Examination Management

**ANTIGRAVITY | NATIVE RUST / egui / eframe**

Detailed scope / Phase diagrams / Normal prompt
Full JSON prompt / Test gates / Completion rules




**TASK 05 IS PARTIAL: LIVE PDF AND AUDIT SERVICES STILL REQUIRE CODEX**

# Task 06 at a glance

Patient directory, safe patient detail editing, examination history, OD/OS image association, navigation, session isolation, and truthful backend readiness.

## IMPORTANT BASELINE

Antigravity Task 05 is reported PARTIAL. The frontend report surface, export state machines, scoped dossier export and audit interface exist, with 94 reported GUI tests passing. The live PDF service and audit persistence were reported absent; these are Codex-owned. Task 06 must not pretend those gaps were completed.

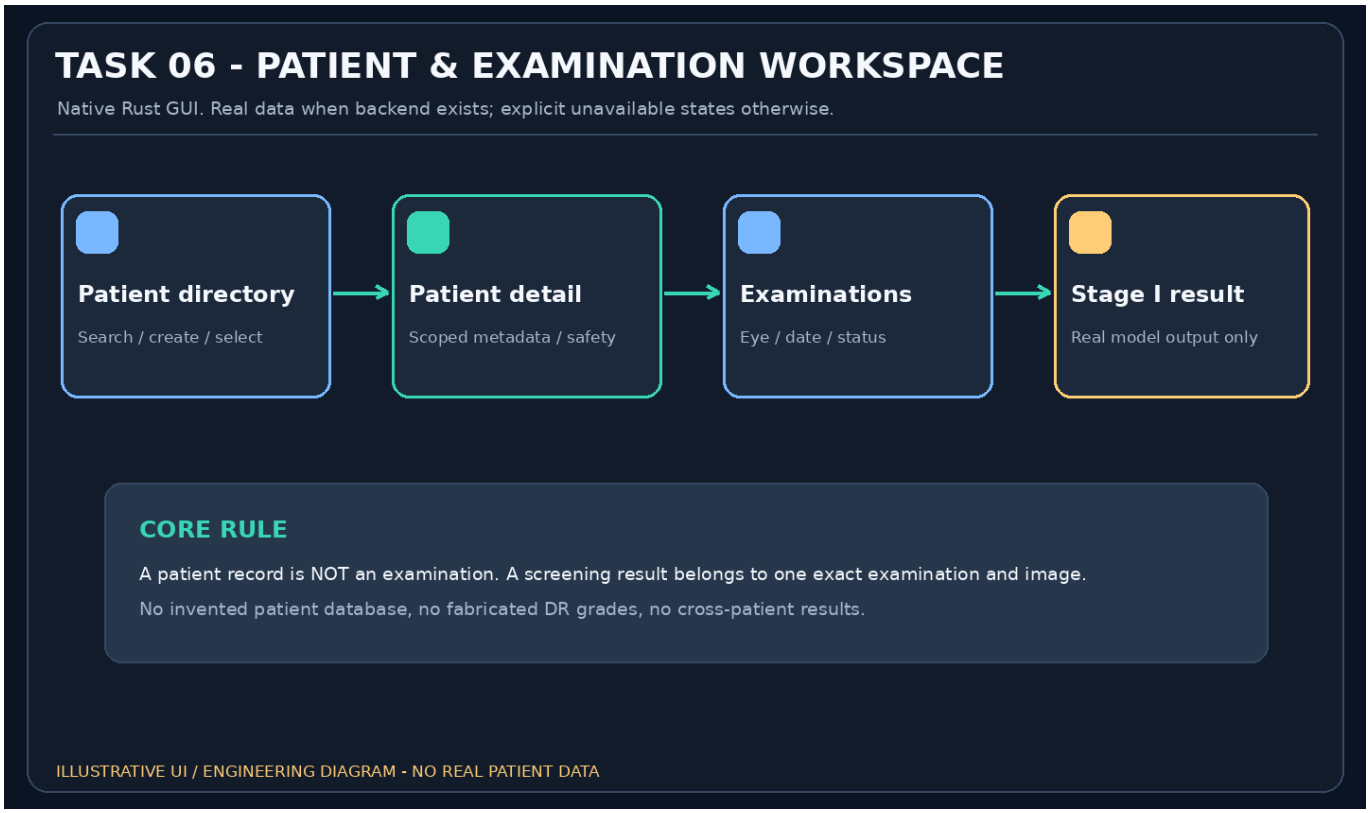
| Workstream | Reported at handoff | Task 06 treatment |
|---|---|---|
| GUI Tasks 01-04 | Reported implemented | Preserve and regression-test |
| GUI Task 05 report UI | Reported implemented | Reuse without redesign |
| Actual PDF generation | Blocked in Codex | Remain unavailable if still absent |
| Persisted audit ledger | Blocked in Codex | No fake audit events |
| Patient CRUD/exam storage | Must audit live API | Build GUI; gate real persistence |
| Stage II/III clinical AI | Not integrated | No fake grades or overlays |

Working principle: a selected patient, a selected examination, an original retinal image and an inference request must never be conflated. Stable identifiers and stale-event rejection are acceptance criteria.

## What is included

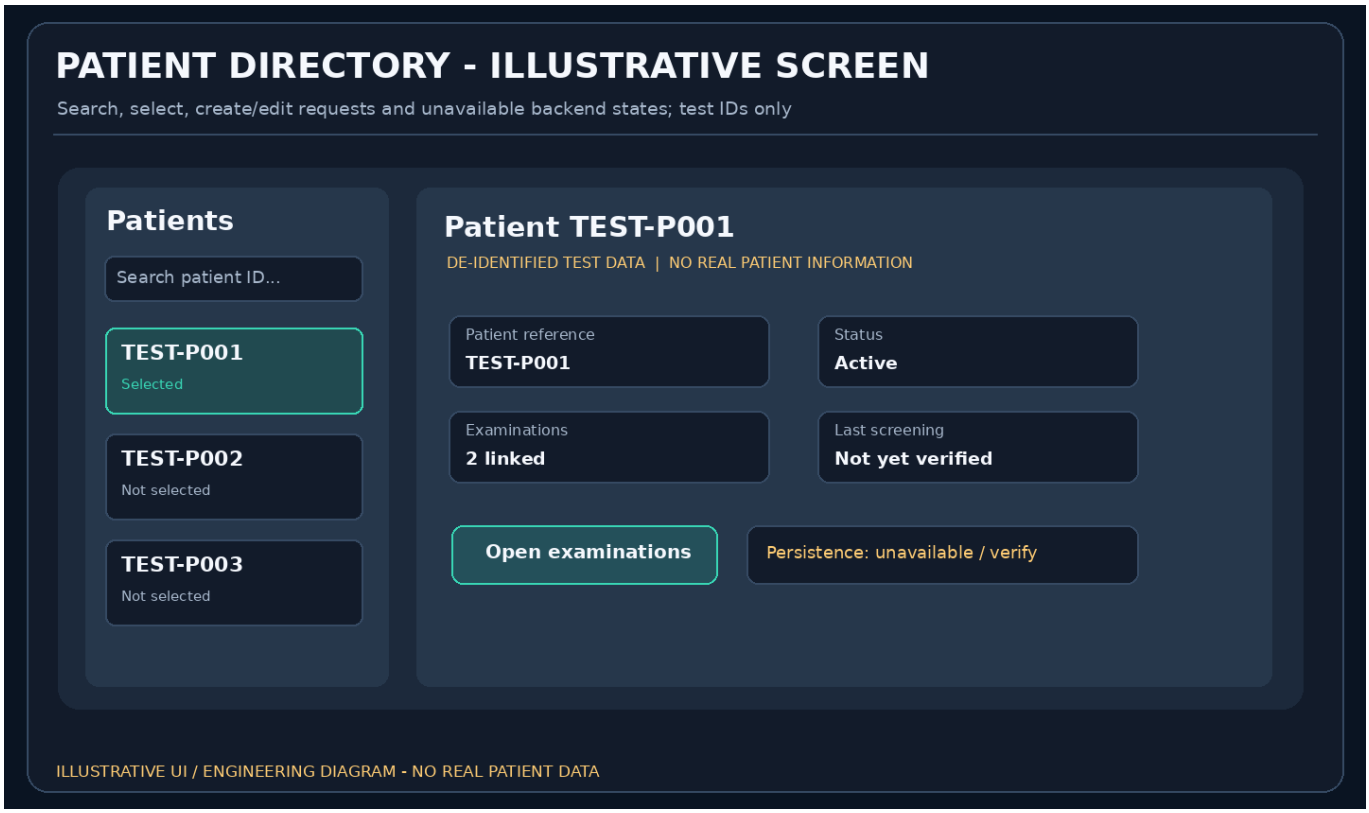
- Six large explanatory phase-specific diagrams, explicitly marked as illustrative test-only UI
- An ordered 22-section normal text prompt, included in full
- A complete JSON master prompt with implementation phases and gates, included in full
- Rust test plan, verification commands, screenshots and strict acceptance criteria
- Preserve existing Task 05 report UX and note unavailable Codex PDF/audit APIs

## Phase diagram 1 - Task 06 whole workflow



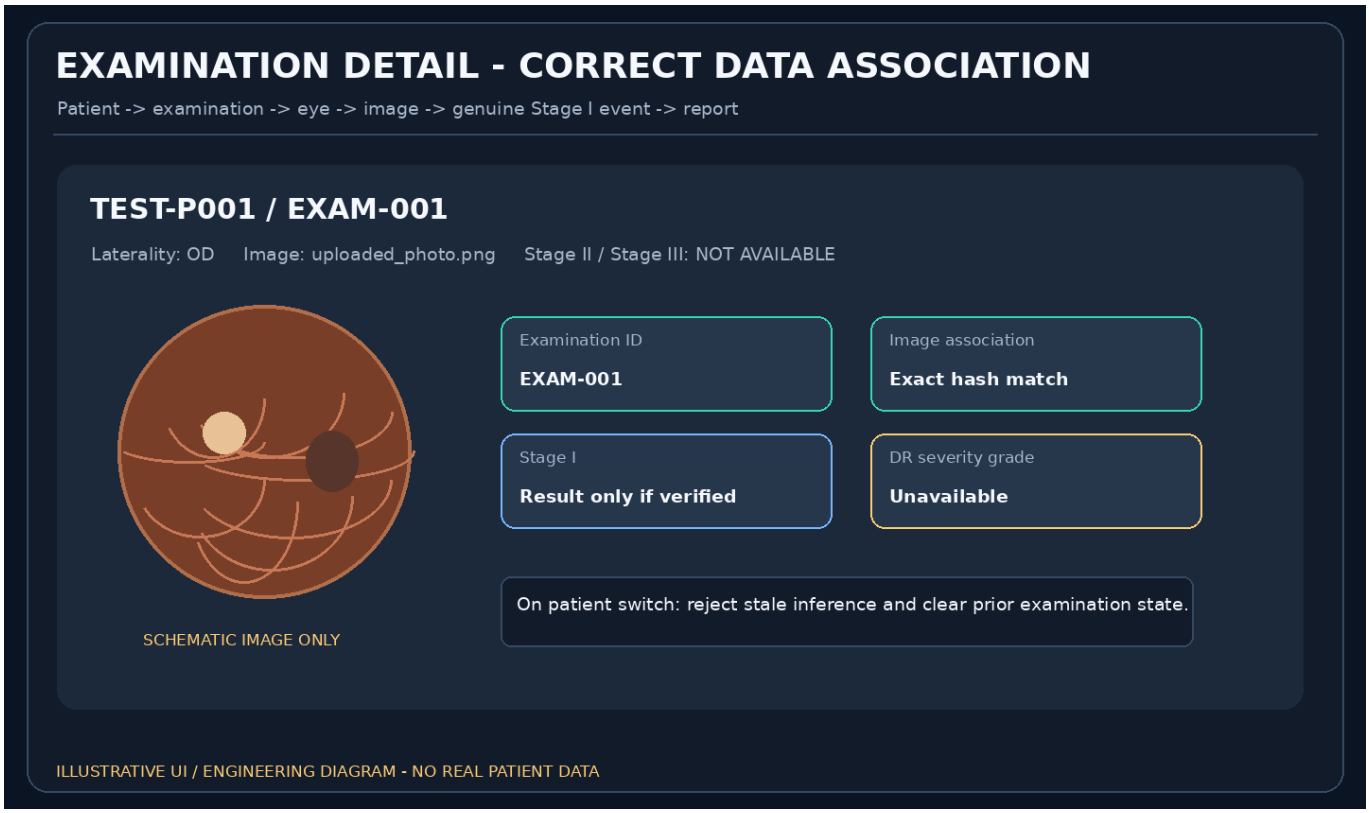

From directory to exact patient and examination, then optionally to genuine Stage I screening. These are application views, not separate trained medical AI stages.

## Phase diagram 2 - Patient directory



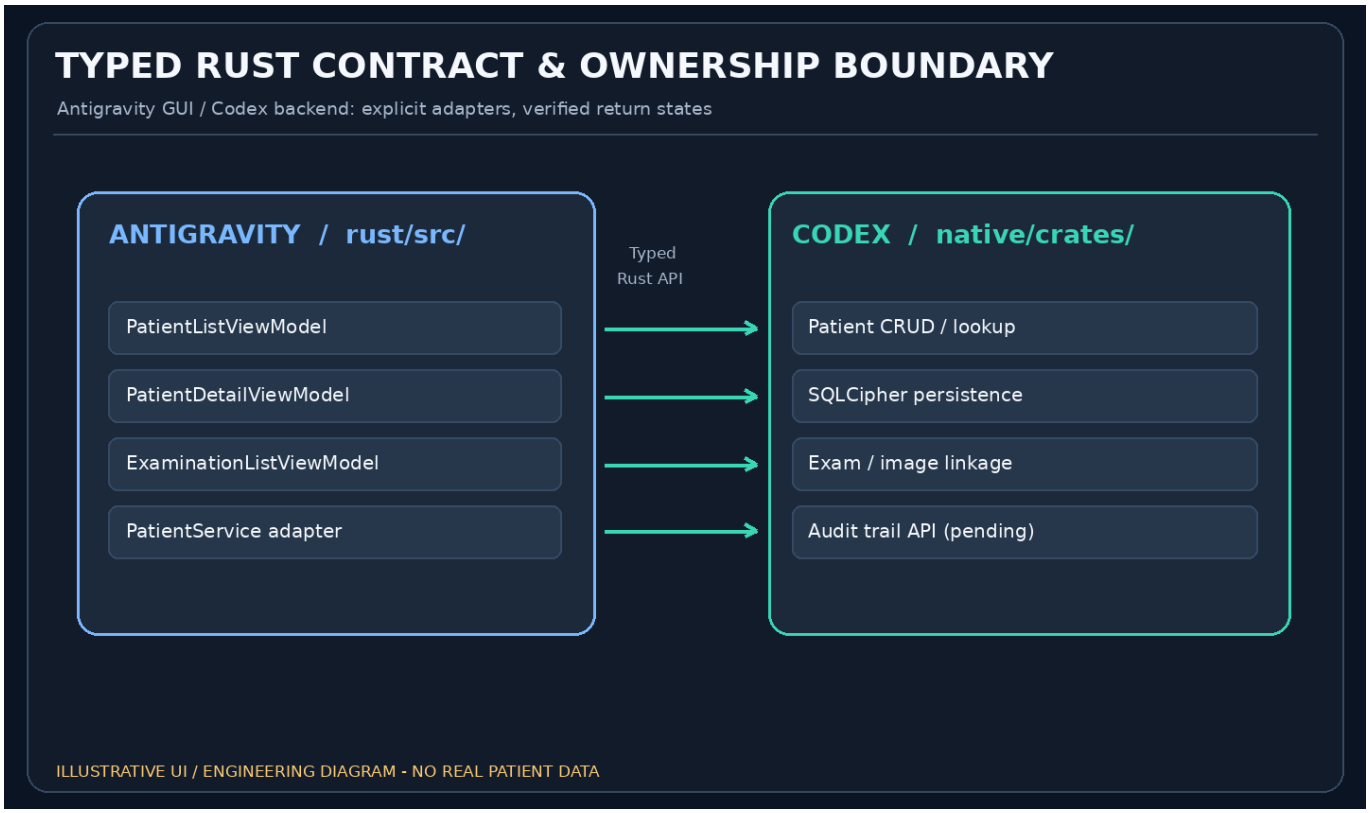

Illustrative GUI composition. TEST-P001 and other values are synthetic identifiers for tests only. Do not fabricate production records.

## Phase diagram 3 - Examination context



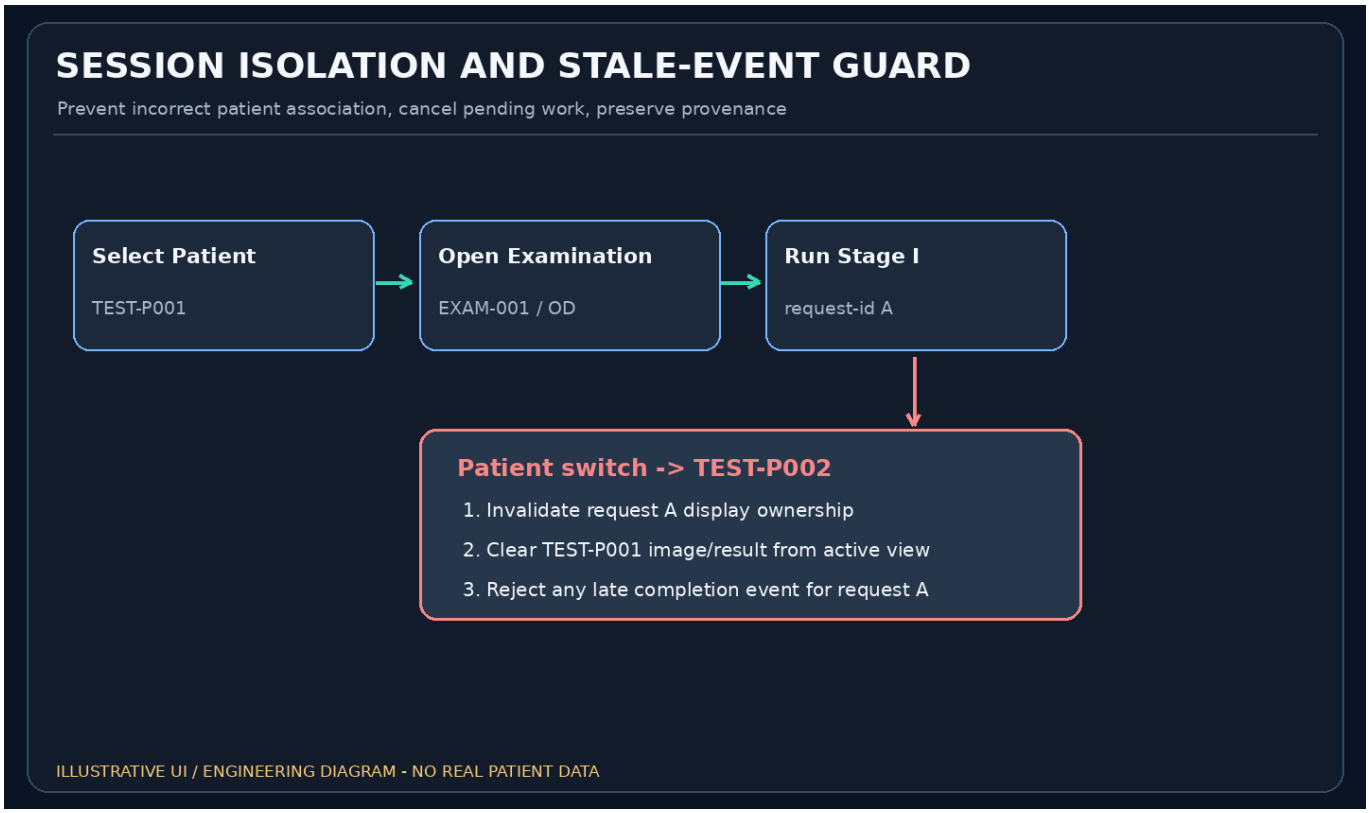

The fundus-like picture is a schematic only. Stage I results must come from real Codex events; DR grading is unavailable without Stage III.

## Phase diagram 4 - Rust ownership and typed interfaces




Antigravity may write GUI adapters. Codex alone implements persistence, audit storage, model inference and backend PDF.

## Phase diagram 5 - Isolation on patient switch




Asynchronous completion for an old request may arrive after the user changes patient. The GUI must reject that event and remove old patient-specific content.

## Phase diagram 6 - Test and acceptance map




Automated logic tests, real desktop screenshots and genuine backend storage tests are separate evidence categories.

# Task 06: implementation playbook

Use this guide as a detailed checklist. The codebase and current backend APIs must be audited before relying on a report.

## Build order

Audit baseline and repo branches -> typed service surface -> patient directory -> patient detail -> examination list/detail -> image linkage -> isolation -> backend unavailable states -> native verification -> docs and handoff.

## Ownership

Antigravity edits GUI code only. Codex edits SQLCipher, Rust backend, original model runtime, medical report engine and audit tables. No force merges.

## Security

No real patient data in screenshots; no plaintext unapproved patient database; never show the wrong patient's image or result; no invented clinical grade.

## Task 05 carryover

A status report with 94 GUI tests does not prove that PDF or audit persistence exists. Preserve truthful unavailable states, note exact Codex API gaps.

## Acceptance strategy

Task 06 GUI read/view flows can be verified independently; live create/update/delete only counts when the backend confirms durable operation and a reload reproduces the record.

# Prompt A / Full normal text for Antigravity

Copy the entire text from the companion TXT file, or copy these full PDF sections. Every instruction below is part of the executable coding brief.

## 00 | YOUR EXACT ASSIGNMENT

You are Antigravity, the principal native Rust GUI engineer for RetinaCare. Implement GUI Phase 1 Task 06: Patient & Examination Management. This is a REAL CODING TASK in the existing egui/eframe desktop project.

Audit, research, plan, implement, run native desktop verification, test, document and prepare Task 07 handoff. Do not stop after planning. Do not begin Task 07.

The main goal is a usable and safe UI for patients, examinations and retinal image association, not an invented electronic medical record system.

## 01 | VERIFIED VS REPORTED BASELINE

The most recent Antigravity Task 05 report says report overview, local PDF export UI, a patient-scoped session dossier export, audit UI contracts and state machines were implemented. It reports 94 Rust GUI tests passing and production PDF/audit backend functions still missing. These are AGENT-REPORTED RESULTS, not independent verification.

Re-audit the actual source. PDF generation in retinacare-reports and audit persistence in retinacare-persistence are reportedly unavailable. Do not represent their UI mock services as real backend behavior.

Stage I is a screening gate only; Stage II lesion/vessel and Stage III grading remain unavailable unless new genuine backend evidence exists. Never interpret a screenshot's illustrative score/latency as a reproducible clinical measurement.

## 02 | REPOSITORIES AND OWNERSHIP

Expected GUI project: /home/claude/Desktop/Hackthon dezine/rust (verify the real root and current branch). Codex backend: /home/claude/Desktop/Dird/native/crates (read-only for Antigravity). GUI-owned editable area: rust/src/, GUI Cargo manifest when justified, Task06 feature docs and concise nonconflicting memory.

Never modify native/crates/, models/, ai_engine/, inference, PDF rendering engine or database schema. No force branch merges, reset --hard, git push, blind cherry-picks or replacing dirty files. Check workspace path dependencies to Dird, especially portability.

Native Rust only: egui/eframe; no HTML, CSS, JavaScript, React, WebView, Electron, Tauri or new web server.

## 03 | MEMORY-FIRST AND SKILLS

Read AGENTS.md, existing Obsidian memory/00-index.md, memory/active-phase.md, relevant notes, and docs/features/GUI-P05-clinical-report-export/13-task06-handoff.md first. Consult existing Ponytail and applicable Superpowers skills; do not reclone. Audit targeted source symbols rather than dumping entire files/repository.

Preserve Codex memory updates; avoid overwriting newer backend notes. Document assumptions, decisions and test evidence with Obsidian wikilinks.

## 04 | EXISTING TYPES / API AUDIT

Inspect rust/src/domain.rs, rust/src/main.rs, rust/src/engine/{backend.rs,report.rs,export.rs}, rust/src/ui/{viewer.rs,composer.rs}, plus actual backend public patient/examination/persistence interfaces (read-only). Determine whether a real patient-list/create/update/archive service, examination creation and retrieval API, patient/exam association, SQLCipher store, audit functions and consent metadata actually exist. Find any duplicated patient types.

Do not implement a second plaintext patient database. Verify whether current patient identifiers are synthetic demo values or real records and never convert test data into production data.

## 05 | FEATURE DOCUMENTS & PLANNING

Create docs/features/GUI-P06-patient-examinations/ with README.md; 01-current-state-audit.md; 02-clinical-workflow-research.md; 03-requirements.md; 04-implementation-plan.md; 05-data-contracts.md; 06-privacy-and-isolation.md; 07-view-state-machines.md; 08-test-plan.md; 09-test-results.md; 10-desktop-verification.md; 11-file-change-report.md; 12-known-blockers.md; 13-codex-api-handoff.md; 14-task07-handoff.md. Reuse existing equivalents if present. Only document actual findings, not filler.

Plan the smallest complete GUI-side changes and move directly to coding.

## 06 | PATIENT DIRECTORY VIEW

Implement a compact, polished dark native Rust patient directory. Features: safe patient-reference display, explicit selection, search by permitted identifiers, optional filters, zero-results, loading, error, disconnected and permission-denied states, and a new-patient request form only when supported. Avoid showing actual names, dates of birth, addresses or diagnoses in default test screenshots.

List item must not show a Stage I grade as if it were a patient-level diagnosis. Make UI robust at small desktop sizes, preserve sidebar and Tasks01-05 visual style. Never ship a production-screen list of invented patients.

Deterministic synthetic patient fixtures allowed ONLY in tests/dev mode and clearly labeled.

## 07 | PATIENT DETAIL & SAFE EDITING

Create detail screen with stable patient reference and only fields supported by verified backend. Inspect permissions/consent fields before displaying them. Edit with proper validation, dirty-field tracking, cancel/revert, save-in-progress, failure recovery, and explicit confirmation for irreversible changes.

A UI save must not say SAVED unless persistence acknowledges success. If backend CRUD is unavailable, disable or label save as unavailable and generate an exact Codex API handoff. No local unencrypted database, automatic network sync or fabricated audit log.

## 08 | EXAMINATION LIST & DETAIL

For the selected patient, show examinations with exact examination ID, laterality OD/OS/unknown, captured timestamp where available, image count/reference, genuine analysis status and report availability. Add UI for creating an examination only with a verified backend API; otherwise implement a tested front-end contract and truthful unavailable state. Separate patient ID, examination ID, image artifact ID and inference request ID.

Support switching, retry on fetch failure, and navigation to original fundus viewer/report without importing a different patient's data. The same patient's OD and OS images must remain distinguishable.

## 09 | RETINAL IMAGE ASSOCIATION

Reuse Task03 viewer and Task01 attachment pipeline. When linking image to exam, validate supported JPEG/PNG formats, image bounds, ownership, source hash if available, and laterality when explicitly supplied. Do not infer OD/OS from appearance.

Ask for explicit confirmation before replacing an image. Reject duplicates or stale image callbacks when relevant. Do not upload images to external services.

Ensure a subsequent Stage I inference result is associated with the exact exam ID, image identity and request ID, not whichever patient is currently selected.

## 10 | NAVIGATION & SESSION LIFECYCLE

Build clear navigation Directory -> Patient -> Examination -> Report (with return path and preserved selection when safe). On patient/exam switch, invalidate pending GUI requests, clear sensitive old patient-specific contents, prevent stale async events from repopulating wrong view, and handle cancellation truthfully. Preserve unsaved form drafts or prompt before discarding them.

Avoid unbounded per-patient in-memory image caches. Cross-patient isolation is a non-negotiable test gate.

## 11 | BACKEND INTERFACE AND ADAPTERS

Prefer existing backend public Rust types and path dependencies; implement narrow GUI-only adapters. Suggested conceptual contracts: PatientQuery, PatientSummary, PatientRecordView, ExaminationSummary, ExaminationDetail, PatientDirectoryEvent, ExaminationEvent, PatientServiceError. Inspect real API and avoid claiming these types exist.

Implement genuine provider when callable, with bounded nonblocking communication; otherwise implement UnavailablePatientService with clear GUI messages. MockPatientService is permissible in #[cfg(test)] only; never route mock patient records into production. Document exactly which Codex service methods are missing, request/response/error shapes and isolation contracts.

## 12 | REPORT / PDF / AUDIT BOUNDARY

Task05 is PARTIAL. Preserve ReportOverview, export modal and audit drawer but do not silently finish or relabel them. If Codex PDF/audit APIs remain absent, show truthful unavailable states.

Do not implement an independent PDF engine or SQLCipher database inside the GUI. Review Task05 export_scoped_patient_session(): if it produces unencrypted JSON, explicitly label it plaintext sensitive export; require intentional user action, secure destination warnings and scope tests; never call plaintext output an encrypted dossier. Do not expose fake audit history.

Provide separate Codex backend API requirements without modifying Codex crates.

## 13 | VISUAL QUALITY

Follow the existing minimal charcoal GUI design, compact professional typography, consistent spacing and keyboard focus. Patient directory, details and exam list should appear native and coherent with Home/Chat/Report. Support window sizes 680x580, 1024x720, 1280x820 and 1600x900 with scrolling rather than clipping.

Show current navigation context; use small indicators, tooltips and disabled explanations. Avoid oversized headers, unrelated redesign, fake charts, decorative diagnoses or unrealistic empty-state metrics. Add real screenshot verification with synthetic nonclinical test fixtures or blank states.

## 14 | PRIVACY, SECURITY & CLINICAL INTEGRITY

Treat patient and retinal data as sensitive. No plaintext logging of full records, no accidental filesystem path disclosure, no external transmission, no misleading claims of HIPAA/FIPS certification. Use backend-owned encryption for durable storage, never new ad hoc GUI storage.

Each operation must be scoped to stable patient_id and examination_id; prohibit cross-patient search-result reuse, image linking and export. Use OS-safe file metadata. Never fabricate DR Grade 0-4, lesion counts, Grad-CAM, model readiness or diagnosis.

Stage I score is not a calibrated clinical probability unless documented. If permissions/audit are unavailable, display that limitation.

## 15 | REQUIRED AUTOMATED TESTS

Run initial tests as baseline then new tests for: empty directory, error and loading states; deterministic search/filter; stable patient selection; invalid ID/field validation; save/cancel paths; unavailable CRUD and truthful status; examination list association and OD/OS; incorrect patient/exam ID rejection; duplicate image attachment; patient switch clears old image/result; late events discarded; cancellation/retry; stale GUI state; navigation return; small window state. Regression test Task01 Home, Task02 Chat, Task03 quad viewer, Task04 Stage I bridge, Task05 Report/export UI. Use synthetic fixtures only in unit tests or clearly labeled dev mode.

## 16 | BUILD & REAL DESKTOP VERIFICATION

From GUI workspace run cargo fmt --all --check; cargo clippy --workspace --all-targets -- -D warnings; cargo test --workspace; cargo build --release; git diff --check. Also inspect whether path-linked backend dependency builds successfully from GUI workspace. Launch real Ubuntu binary if graphical environment is accessible, navigate to patient directory and examinations; test loading, empty, error/unavailable states and switching; verify no old patient artifacts survive.

Capture screenshots inside Task06 docs: directory_empty.png, directory_test_fixture.png (only in isolated dev mode), patient_detail.png, exams_list.png, backend_unavailable.png, small_window.png. Do not claim a real database or save operation worked unless directly observed. Record exact commands/exit codes, environment and test count.

## 17 | PERFORMANCE & CONCURRENCY

No database queries, image decode or patient record materialization inside egui paint loop. Use existing nonblocking event channels/worker threads, bounded queues and correlated requests. Debounce filtering only when needed; avoid performance guesses.

Cap list lengths or virtualize when practical, lazily fetch examination details, avoid cloning full-resolution fundus images per frame, and drop stale textures after switching patients. Separate real backend event handling from mock UI tests.

## 18 | HANDOFF TO CODEX

Write docs/features/GUI-P06-patient-examinations/13-codex-api-handoff.md with actual existing backend APIs, missing patient CRUD/search, examination create/list/detail, image binding, persistence, audit and permission requirements. Include minimum typed Rust signatures, data ownership and the exact blocking unit/integration tests. Mark anything not yet implemented.

Do not tell Codex to clone or replace the GUI or alter Antigravity-owned types without coordination.

## 19 | MEMORY & CHANGE REPORT

Update memory/00-index.md, memory/active-phase.md and dedicated GUI-P06 note; update decisions/gotchas only as needed. Do not overwrite Codex memory newer than your snapshot. Every changed file in docs/features/GUI-P06-patient-examinations/11-file-change-report.md must include created/modified, exact functions/types, why, tests, result, other dependencies and any skipped verification.

Keep all notes concise and linked using Obsidian [[wikilinks]]. Do not commit/push without authorization.

## 20 | DEFINITION OF DONE & STATUS RULE

A GUI-only Task06 can be COMPLETE only if directory, patient detail, examination lists, accurate navigation and isolation, loading/empty/error states, backend adapter and verified tests are truly implemented and render correctly. Separately label LIVE PATIENT CRUD, SQLCIPHER PERSISTENCE, AUDIT and REAL EXAMINATION SAVE as BLOCKED/PENDING if Codex has not implemented them. If the intended task promises live durable patient operations and the backend is absent, overall Task06 must be PARTIAL, not COMPLETE.

A unit-test mock is not production storage. No clinical certification is claimed.

## 21 | FINAL REPORT & NO-SKIP GATE

Report A. Git state/source audit; B. baseline report truth table; C.

directory/detail/exam UX; D. typed API and readiness; E. patient/exam/image identity; F.

true backend write/read status; G. privacy/export safety; H. automated tests exact commands; I.

actual native screenshots; J. file-by-file report; K. memory docs; L.

exact Codex prerequisites; M. Task06 verdict COMPLETE/PARTIAL/BLOCKED; N. Task07 handoff.

Do not start Task07 automatically. END OF PROMPT: START IMPLEMENTATION NOW, NOT A PLAN-ONLY ANSWER.

# Prompt B / Full JSON master prompt

This is the full structured master prompt. Its exact machine-readable version is also supplied as a separate .json file.


```
{
  "prompt_id": "RETINACARE-ANTIGRAVITY-GUI-P01-TASK06",
  "project": "RetinaCare",
  "agent": "Antigravity",
  "phase": "GUI Phase 1",
  "task": "06",
  "task_title": "Patient & Examination Management",
  "priority": "P0",
  "execution_mode": "MEMORY_AUDIT_RESEARCH_PLAN_IMPLEMENT_TEST_LAUNCH_DOCUMENT",
  "task05_handoff": {
    "reported_frontend": "Report overview, PDF export UI, scoped session dossier JSON, audit drawer; 94 Rust GUI
 tests",
    "overall": "PARTIAL",
    "backend_blockers": [
      "real PDF engine",
      "audit_events persistence/query API"
    ],
    "medical_availability": "Only genuine Stage I screening has been reported connected; Stage II and Stage III
are unavailable until verified",
    "accuracy": "Agent-reported status, must independently inspect source."
  },
  "repositories": {
    "gui_expected": "/home/claude/Desktop/Hackthon dezine/rust",
    "backend_read_only": "/home/claude/Desktop/Dird/native/crates",
    "rule": "Verify actual roots/branches/worktree; never assume report paths or versions."
  },
  "ownership": {
    "editable": "GUI-owned rust/src/, justified rust/Cargo.toml, GUI-P06 docs, nonconflicting Obsidian memory",
    "prohibited": [
      "native/crates/",
      "ai_engine/",
      "models/",
      "backend SQLCipher schema",
      "ONNX weights/runtime",
      "Codex report engine",
      "forcing Git merges or changing branches destructively",
      "git push or commit without authorization"
    ]
  },
  "technology": {
    "production": "Rust egui/eframe only",
    "integration": "Existing public Rust crates, typed nonblocking adapters",
    "forbidden": [
      "HTML",
      "CSS",
      "JavaScript",
      "TypeScript",
      "React",
      "Tauri",
      "Electron",
      "WebView",
      "HTTP patient service",
      "Python production UI",
      "new plaintext patient database"
    ]
  },
  "read_first": [
```

```
    "AGENTS.md",
    "memory/00-index.md",
    "memory/active-phase.md",
    "existing Ponytail and relevant Superpowers skill instructions",
    "docs/features/GUI-P05-clinical-report-export/13-task06-handoff.md",
    "rust/src/domain.rs",
    "rust/src/main.rs",
    "rust/src/engine/backend.rs",
    "rust/src/engine/report.rs",
    "rust/src/engine/export.rs",
    "rust/src/ui/viewer.rs",
    "read-only actual backend domain/persistence APIs"
  ],
  "implementation_phases": [
    {
      "id": "00",
      "title": "YOUR EXACT ASSIGNMENT",
      "instructions": "You are Antigravity, the principal native Rust GUI engineer for RetinaCare. Implement GUI
 Phase 1 Task 06: Patient & Examination Management. This is a REAL CODING TASK in the existing egui/eframe
desktop project. Audit, research, plan, implement, run native desktop verification, test, document and prepare
Task 07 handoff. Do not stop after planning. Do not begin Task 07. The main goal is a usable and safe UI for
patients, examinations and retinal image association, not an invented electronic medical record system."
    },
    {
      "id": "01",
      "title": "VERIFIED VS REPORTED BASELINE",
      "instructions": "The most recent Antigravity Task 05 report says report overview, local PDF export UI, a
patient-scoped session dossier export, audit UI contracts and state machines were implemented. It reports 94
Rust GUI tests passing and production PDF/audit backend functions still missing. These are AGENT-REPORTED
RESULTS, not independent verification. Re-audit the actual source. PDF generation in retinacare-reports and
audit persistence in retinacare-persistence are reportedly unavailable. Do not represent their UI mock services
as real backend behavior. Stage I is a screening gate only; Stage II lesion/vessel and Stage III grading remain
unavailable unless new genuine backend evidence exists. Never interpret a screenshot's illustrative
score/latency as a reproducible clinical measurement."
    },
    {
      "id": "02",
      "title": "REPOSITORIES AND OWNERSHIP",
      "instructions": "Expected GUI project: /home/claude/Desktop/Hackthon dezine/rust (verify the real root and
 current branch). Codex backend: /home/claude/Desktop/Dird/native/crates (read-only for Antigravity). GUI-owned
editable area: rust/src/, GUI Cargo manifest when justified, Task06 feature docs and concise nonconflicting
memory. Never modify native/crates/, models/, ai_engine/, inference, PDF rendering engine or database schema. No
 force branch merges, reset --hard, git push, blind cherry-picks or replacing dirty files. Check workspace path
dependencies to Dird, especially portability. Native Rust only: egui/eframe; no HTML, CSS, JavaScript, React,
WebView, Electron, Tauri or new web server."
    },
    {
      "id": "03",
      "title": "MEMORY-FIRST AND SKILLS",
      "instructions": "Read AGENTS.md, existing Obsidian memory/00-index.md, memory/active-phase.md, relevant
notes, and docs/features/GUI-P05-clinical-report-export/13-task06-handoff.md first. Consult existing Ponytail
and applicable Superpowers skills; do not reclone. Audit targeted source symbols rather than dumping entire
files/repository. Preserve Codex memory updates; avoid overwriting newer backend notes. Document assumptions,
decisions and test evidence with Obsidian wikilinks."
    },
    {
      "id": "04",
      "title": "EXISTING TYPES / API AUDIT",
      "instructions": "Inspect rust/src/domain.rs, rust/src/main.rs,
rust/src/engine/{backend.rs,report.rs,export.rs}, rust/src/ui/{viewer.rs,composer.rs}, plus actual backend
public patient/examination/persistence interfaces (read-only). Determine whether a real
patient-list/create/update/archive service, examination creation and retrieval API, patient/exam association,
SQLCipher store, audit functions and consent metadata actually exist. Find any duplicated patient types. Do not
implement a second plaintext patient database. Verify whether current patient identifiers are synthetic demo
values or real records and never convert test data into production data."
    },
    {
      "id": "05",
      "title": "FEATURE DOCUMENTS & PLANNING",
```

```
      "instructions": "Create docs/features/GUI-P06-patient-examinations/ with README.md;
01-current-state-audit.md; 02-clinical-workflow-research.md; 03-requirements.md; 04-implementation-plan.md;
05-data-contracts.md; 06-privacy-and-isolation.md; 07-view-state-machines.md; 08-test-plan.md;
09-test-results.md; 10-desktop-verification.md; 11-file-change-report.md; 12-known-blockers.md;
13-codex-api-handoff.md; 14-task07-handoff.md. Reuse existing equivalents if present. Only document actual
findings, not filler. Plan the smallest complete GUI-side changes and move directly to coding."
    },
    {
      "id": "06",
      "title": "PATIENT DIRECTORY VIEW",
      "instructions": "Implement a compact, polished dark native Rust patient directory. Features: safe
patient-reference display, explicit selection, search by permitted identifiers, optional filters, zero-results,
loading, error, disconnected and permission-denied states, and a new-patient request form only when supported.
Avoid showing actual names, dates of birth, addresses or diagnoses in default test screenshots. List item must
not show a Stage I grade as if it were a patient-level diagnosis. Make UI robust at small desktop sizes,
preserve sidebar and Tasks01-05 visual style. Never ship a production-screen list of invented patients.
Deterministic synthetic patient fixtures allowed ONLY in tests/dev mode and clearly labeled."
    },
    {
      "id": "07",
      "title": "PATIENT DETAIL & SAFE EDITING",
      "instructions": "Create detail screen with stable patient reference and only fields supported by verified
backend. Inspect permissions/consent fields before displaying them. Edit with proper validation, dirty-field
tracking, cancel/revert, save-in-progress, failure recovery, and explicit confirmation for irreversible changes.
 A UI save must not say SAVED unless persistence acknowledges success. If backend CRUD is unavailable, disable
or label save as unavailable and generate an exact Codex API handoff. No local unencrypted database, automatic
network sync or fabricated audit log."
    },
    {
      "id": "08",
      "title": "EXAMINATION LIST & DETAIL",
      "instructions": "For the selected patient, show examinations with exact examination ID, laterality
OD/OS/unknown, captured timestamp where available, image count/reference, genuine analysis status and report
availability. Add UI for creating an examination only with a verified backend API; otherwise implement a tested
front-end contract and truthful unavailable state. Separate patient ID, examination ID, image artifact ID and
inference request ID. Support switching, retry on fetch failure, and navigation to original fundus viewer/report
 without importing a different patient's data. The same patient's OD and OS images must remain distinguishable."
    },
    {
      "id": "09",
      "title": "RETINAL IMAGE ASSOCIATION",
      "instructions": "Reuse Task03 viewer and Task01 attachment pipeline. When linking image to exam, validate
supported JPEG/PNG formats, image bounds, ownership, source hash if available, and laterality when explicitly
supplied. Do not infer OD/OS from appearance. Ask for explicit confirmation before replacing an image. Reject
duplicates or stale image callbacks when relevant. Do not upload images to external services. Ensure a
subsequent Stage I inference result is associated with the exact exam ID, image identity and request ID, not
whichever patient is currently selected."
    },
    {
      "id": "10",
      "title": "NAVIGATION & SESSION LIFECYCLE",
      "instructions": "Build clear navigation Directory -> Patient -> Examination -> Report (with return path
and preserved selection when safe). On patient/exam switch, invalidate pending GUI requests, clear sensitive old
 patient-specific contents, prevent stale async events from repopulating wrong view, and handle cancellation
truthfully. Preserve unsaved form drafts or prompt before discarding them. Avoid unbounded per-patient in-memory
 image caches. Cross-patient isolation is a non-negotiable test gate."
    },
    {
      "id": "11",
      "title": "BACKEND INTERFACE AND ADAPTERS",
      "instructions": "Prefer existing backend public Rust types and path dependencies; implement narrow
GUI-only adapters. Suggested conceptual contracts: PatientQuery, PatientSummary, PatientRecordView,
ExaminationSummary, ExaminationDetail, PatientDirectoryEvent, ExaminationEvent, PatientServiceError. Inspect
real API and avoid claiming these types exist. Implement genuine provider when callable, with bounded
nonblocking communication; otherwise implement UnavailablePatientService with clear GUI messages.
MockPatientService is permissible in #[cfg(test)] only; never route mock patient records into production.
Document exactly which Codex service methods are missing, request/response/error shapes and isolation
contracts."
    },
```

```
    {
      "id": "12",
      "title": "REPORT / PDF / AUDIT BOUNDARY",
      "instructions": "Task05 is PARTIAL. Preserve ReportOverview, export modal and audit drawer but do not
silently finish or relabel them. If Codex PDF/audit APIs remain absent, show truthful unavailable states. Do not
 implement an independent PDF engine or SQLCipher database inside the GUI. Review Task05
export_scoped_patient_session(): if it produces unencrypted JSON, explicitly label it plaintext sensitive
export; require intentional user action, secure destination warnings and scope tests; never call plaintext
output an encrypted dossier. Do not expose fake audit history. Provide separate Codex backend API requirements
without modifying Codex crates."
    },
    {
      "id": "13",
      "title": "VISUAL QUALITY",
      "instructions": "Follow the existing minimal charcoal GUI design, compact professional typography,
consistent spacing and keyboard focus. Patient directory, details and exam list should appear native and
coherent with Home/Chat/Report. Support window sizes 680x580, 1024x720, 1280x820 and 1600x900 with scrolling
rather than clipping. Show current navigation context; use small indicators, tooltips and disabled explanations.
 Avoid oversized headers, unrelated redesign, fake charts, decorative diagnoses or unrealistic empty-state
metrics. Add real screenshot verification with synthetic nonclinical test fixtures or blank states."
    },
    {
      "id": "14",
      "title": "PRIVACY, SECURITY & CLINICAL INTEGRITY",
      "instructions": "Treat patient and retinal data as sensitive. No plaintext logging of full records, no
accidental filesystem path disclosure, no external transmission, no misleading claims of HIPAA/FIPS
certification. Use backend-owned encryption for durable storage, never new ad hoc GUI storage. Each operation
must be scoped to stable patient_id and examination_id; prohibit cross-patient search-result reuse, image
linking and export. Use OS-safe file metadata. Never fabricate DR Grade 0-4, lesion counts, Grad-CAM, model
readiness or diagnosis. Stage I score is not a calibrated clinical probability unless documented. If
permissions/audit are unavailable, display that limitation."
    },
    {
      "id": "15",
      "title": "REQUIRED AUTOMATED TESTS",
      "instructions": "Run initial tests as baseline then new tests for: empty directory, error and loading
states; deterministic search/filter; stable patient selection; invalid ID/field validation; save/cancel paths;
unavailable CRUD and truthful status; examination list association and OD/OS; incorrect patient/exam ID
rejection; duplicate image attachment; patient switch clears old image/result; late events discarded;
cancellation/retry; stale GUI state; navigation return; small window state. Regression test Task01 Home, Task02
Chat, Task03 quad viewer, Task04 Stage I bridge, Task05 Report/export UI. Use synthetic fixtures only in unit
tests or clearly labeled dev mode."
    },
    {
      "id": "16",
      "title": "BUILD & REAL DESKTOP VERIFICATION",
      "instructions": "From GUI workspace run cargo fmt --all --check; cargo clippy --workspace --all-targets --
 -D warnings; cargo test --workspace; cargo build --release; git diff --check. Also inspect whether path-linked
backend dependency builds successfully from GUI workspace. Launch real Ubuntu binary if graphical environment is
 accessible, navigate to patient directory and examinations; test loading, empty, error/unavailable states and
switching; verify no old patient artifacts survive. Capture screenshots inside Task06 docs: directory_empty.png,
 directory_test_fixture.png (only in isolated dev mode), patient_detail.png, exams_list.png,
backend_unavailable.png, small_window.png. Do not claim a real database or save operation worked unless directly
 observed. Record exact commands/exit codes, environment and test count."
    },
    {
      "id": "17",
      "title": "PERFORMANCE & CONCURRENCY",
      "instructions": "No database queries, image decode or patient record materialization inside egui paint
loop. Use existing nonblocking event channels/worker threads, bounded queues and correlated requests. Debounce
filtering only when needed; avoid performance guesses. Cap list lengths or virtualize when practical, lazily
fetch examination details, avoid cloning full-resolution fundus images per frame, and drop stale textures after
switching patients. Separate real backend event handling from mock UI tests."
    },
    {
      "id": "18",
      "title": "HANDOFF TO CODEX",
      "instructions": "Write docs/features/GUI-P06-patient-examinations/13-codex-api-handoff.md with actual
existing backend APIs, missing patient CRUD/search, examination create/list/detail, image binding, persistence,
```

```
audit and permission requirements. Include minimum typed Rust signatures, data ownership and the exact blocking
unit/integration tests. Mark anything not yet implemented. Do not tell Codex to clone or replace the GUI or
alter Antigravity-owned types without coordination."
    },
    {
      "id": "19",
      "title": "MEMORY & CHANGE REPORT",
      "instructions": "Update memory/00-index.md, memory/active-phase.md and dedicated GUI-P06 note; update
decisions/gotchas only as needed. Do not overwrite Codex memory newer than your snapshot. Every changed file in
docs/features/GUI-P06-patient-examinations/11-file-change-report.md must include created/modified, exact
functions/types, why, tests, result, other dependencies and any skipped verification. Keep all notes concise and
 linked using Obsidian [[wikilinks]]. Do not commit/push without authorization."
    },
    {
      "id": "20",
      "title": "DEFINITION OF DONE & STATUS RULE",
      "instructions": "A GUI-only Task06 can be COMPLETE only if directory, patient detail, examination lists,
accurate navigation and isolation, loading/empty/error states, backend adapter and verified tests are truly
implemented and render correctly. Separately label LIVE PATIENT CRUD, SQLCIPHER PERSISTENCE, AUDIT and REAL
EXAMINATION SAVE as BLOCKED/PENDING if Codex has not implemented them. If the intended task promises live
durable patient operations and the backend is absent, overall Task06 must be PARTIAL, not COMPLETE. A unit-test
mock is not production storage. No clinical certification is claimed."
    },
    {
      "id": "21",
      "title": "FINAL REPORT & NO-SKIP GATE",
      "instructions": "Report A. Git state/source audit; B. baseline report truth table; C.
directory/detail/exam UX; D. typed API and readiness; E. patient/exam/image identity; F. true backend write/read
 status; G. privacy/export safety; H. automated tests exact commands; I. actual native screenshots; J.
file-by-file report; K. memory docs; L. exact Codex prerequisites; M. Task06 verdict COMPLETE/PARTIAL/BLOCKED;
N. Task07 handoff. Do not start Task07 automatically. END OF PROMPT: START IMPLEMENTATION NOW, NOT A PLAN-ONLY
ANSWER."
    }
  ],
  "feature_documents": {
    "folder": "docs/features/GUI-P06-patient-examinations/",
    "files": [
      "README.md",
      "01-current-state-audit.md",
      "02-clinical-workflow-research.md",
      "03-requirements.md",
      "04-implementation-plan.md",
      "05-data-contracts.md",
      "06-privacy-and-isolation.md",
      "07-view-state-machines.md",
      "08-test-plan.md",
      "09-test-results.md",
      "10-desktop-verification.md",
      "11-file-change-report.md",
      "12-known-blockers.md",
      "13-codex-api-handoff.md",
      "14-task07-handoff.md"
    ]
  },
  "mandatory_verification": {
    "commands": [
      "cargo fmt --all --check",
      "cargo clippy --workspace --all-targets -- -D warnings",
      "cargo test --workspace",
      "cargo build --release",
      "git diff --check"
    ],
    "workdir": "rust/",
    "checks": [
      "Patient list/search/filter UX",
      "Patient detail validation",
      "Patient/exam linkage",
      "OD/OS association",
      "No stale cross-patient data",
```

```
      "Mock never used in production",
      "Unavailable CRUD truthful",
      "No silent plaintext persistence",
      "Task01-Task05 regression",
      "Real desktop screenshots when possible"
    ]
  },
  "completion_contract": {
    "status_values": [
      "COMPLETE",
      "PARTIAL",
      "BLOCKED"
    ],
    "rules": [
      "Do not claim live CRUD without actual persisted backend operation and reload test",
      "Do not claim SQLCipher/audit/PDF engine if APIs are unavailable",
      "Mark overall PARTIAL if live patient operations are a mandatory feature but backend APIs missing",
      "Never present synthetic patient details as real production records",
      "Do not start Task07"
    ],
    "handoff": "Document exact minimal Codex backend API additions and Task07 prerequisites"
  },
  "final_instruction": "ANTIGRAVITY: READ MEMORY FIRST. VERIFY TASK05 PARTIAL STATE. BUILD GUI TASK06 PATIENT
DIRECTORY/DETAIL/EXAMINATION SCREENS AND STRONG IDENTITY ISOLATION IN NATIVE RUST WITHOUT TOUCHING CODEX
BACKEND. IMPLEMENT AND TEST ACTUAL AVAILABLE BEHAVIOR, REPORT MISSING PERSISTENCE HONESTLY, VERIFY DESKTOP AND
ALL REGRESSION TESTS, UPDATE OBISIDIAN MEMORY AND FILE-CHANGE REPORT. DO NOT START TASK07 OR CLAIM UNSUPPORTED
COMPLETION."
}
```

# 100% acceptance / Task 06 gate

### [Directory]

A real patient directory view exists, searchable/filterable on allowed fields, with verified empty/loading/error states.

### [Patient Detail]

Selected patient identity and supported metadata are displayed, and editing is truthful about whether persistence exists.

### [Examinations]

List/detail correctly scopes all exams to the active patient and handles OD/OS and image identity.

### [Isolation]

Switching patient/examination clears old context; stale results and cross-patient requests are rejected.

### [Backend]

Genuine existing API is used when available; unavailable patient CRUD never silently succeeds.

### [Privacy]

No fabricated production patient records, no undocumented plaintext durable store, no fake compliance statements.

### [Regressions]

Home, Chat, Viewer, Stage I GUI bridge, Task05 Report/export UI continue to pass.

### [Quality]

Format, strict Clippy, workspace tests, release build and git diff checks pass.

### [Desktop]

Actual native Linux screenshots of Task06 states are reviewed at multiple window sizes.

### [Documentation]

Full file change report, Codex API handoff, tests and Obsidian memory updated; Task07 not begun.

## Verdict policy

Report COMPONENT-LEVEL GUI readiness separately from LIVE PATIENT DATA PERSISTENCE. Mark overall PARTIAL if intended live CRUD or audit/PDF dependencies remain unavailable. The feature is not clinically certified.

Reference: user-supplied Task05 implementation report, latest available in this conversation. All Task06 diagrams are conceptual, not screenshots of the actual application.